

14_For_Loop

```
fruits=["apple","banana","cherry"] #Lists conatining frutis
print(fruits)
print(type(fruits))
```

[2] ✓ 0.0s

```
... ['apple', 'banana', 'cherry']
<class 'list'>
```

```
for i in fruits:
    print(fruits)
```

[3] ✓ 0.0s

```
... ['apple', 'banana', 'cherry']
['apple', 'banana', 'cherry']
['apple', 'banana', 'cherry']
```

```
for i in fruits:
    print(i)
```

[4] ✓ 0.0s

```
... apple
banana
cherry
```

▷ ✓

```
# After seeing the above two examples. we can clearly see that below understandings
# for --> keyword indicating the start of the loop
# Iterable --> The sequence or Iterable object being looped for. Example here fruits list is iterable in above example
# variable/Iterator --> It takes on the value of each item in the iterable during each iteration.
```

[]

```
# Range Function
# We can generate a sequence of numbers using range() function.
# Range(10) generates numbers from 0 to 9 (10 numbers).
# We can also define the start, stop and step size as well as per our needs.
# default step size is 1 if we don't specify.
# Range fucntions generally doesn't store all the values in memory as it would be inefficient.
# so it remembers the start, stop and step size and generates the next number on the go.
```

[6] ✓ 0.0s

```
for i in range(5):
    print(i)
```

[7] ✓ 0.0s

```
... 0
1
2
3
4
```

▷ ✓

```
for i in range(0,20,5):
    print(i)
```

[9] ✓ 0.0s

```
... 0
5
10
15
```

```
# Using length and range together instead of index values to fetch iterable values using for loop
print(fruits)
for i in range(len(fruits)):      # here len gives 3 and range(3) gives 0,1,2 and we can use those values as index in each iteration
    print(fruits[i])
```

[12] ✓ 0.0s

```
... ['apple', 'banana', 'cherry']
apple
banana
cherry
```

```
# Enumerate merges direct iterator in for loop and above length and range combo together
# Enumerator generates two items one is index and other is value as Index, Value
print(fruits)
print(enumerate(fruits))      # returns Iterable holding result as Index, Value
for index, value in enumerate(fruits):
    print("At position {}, we have fruit named {}".format(index,value))
```

[15] ✓ 0.0s

```
... ['apple', 'banana', 'cherry']
<enumerate object at 0x7b16d3292520>
At position 0, we have fruit named apple
At position 1, we have fruit named banana
At position 2, we have fruit named cherry
```

```
# For loop with Strings
name="Hanumanth"

for i in name:
    print(i)
```

[16] ✓ 0.0s

```
... H
    a
    n
    u
    m
    a
    n
    t
    h
```

```
# if else condition inside a for loop
nums = [1,2,3,4,5,6]

for n in nums:
    if n%2 == 0:
        print("Number {} is even number".format(n))
    else:
        print("Number {} is odd number".format(n))
```

[17] ✓ 0.0s

```
... Number 1 is odd number
    Number 2 is even number
    Number 3 is odd number
    Number 4 is even number
    Number 5 is odd number
    Number 6 is even number
```

```

students=[{"name":"Hanu","Percentage":90},
           {"name":"Allada","Percentage":80},
           {"name":"Rao","Percentage":70},
           {"name":"Hanumanth", "Percentage":30}]
print(students)
print(type(students))
print(type(students[0]))

```

[22] ✓ 0.0s

```

... [{"name": 'Hanu', 'Percentage': 90}, {'name': 'Allada', 'Percentage': 80}, {'name': 'Rao', 'Percentage': 70}, {'name': 'Hanumanth', 'Percentage': 30}]
<class 'list'>
<class 'dict'>

```

```

# Combination of dictionaries, list, for loop and if else if
for stud in students:
    stud_name=stud["name"]
    stud_Perc=stud["Percentage"]

    if stud_Perc >= 90:
        print(f"{stud_name}% is {stud_Perc} and he is excellent")
    elif stud_Perc >=80 and stud_Perc < 90:
        print(f"{stud_name} % is {stud_Perc} and he is Good")
    elif stud_Perc >= 70 and stud_Perc < 80:
        print(f"{stud_name} % is {stud_Perc} and he is Average")
    else:
        print(f"{stud_name} % is {stud_Perc} and he is Poor")

```

[27] ✓ 0.0s

```

... Hanu% is 90 and he is excellent
    Allada % is 80 and he is Good
    Rao % is 70 and he is Average
    Hanumanth % is 30 and he is Poor

```

```

# Prime numbers falls between given two range of numbers
start_num = -3
end_num = 50

for num in range(start_num, end_num+1): # to include the end_num, added +1 to end_num as range considers before the end_num
    if num > 1:
        prime_cnt=0
        for i in range(2,num):
            if num%i==0:
                prime_cnt += 1
                break # we can keep this to stop if its already divided by other nums to save some iterations/processing time.
            if prime_cnt == 0:
                print(f"{num} is Prime Number")
    else:
        print(f"{num} is negative number or less than Prime minimum value")

```

[42] ✓ 0.0s

```

... -3 is negative number or less than Prime minimum value
    -2 is negative number or less than Prime minimum value
    -1 is negative number or less than Prime minimum value
    0 is negative number or less than Prime minimum value
    1 is negative number or less than Prime minimum value
    2 is Prime Number
    3 is Prime Number
    5 is Prime Number
    7 is Prime Number
    11 is Prime Number
    13 is Prime Number
    17 is Prime Number
    19 is Prime Number
    23 is Prime Number
    29 is Prime Number
    31 is Prime Number
    37 is Prime Number
    41 is Prime Number
    43 is Prime Number
    47 is Prime Number

```